Universität Rostock



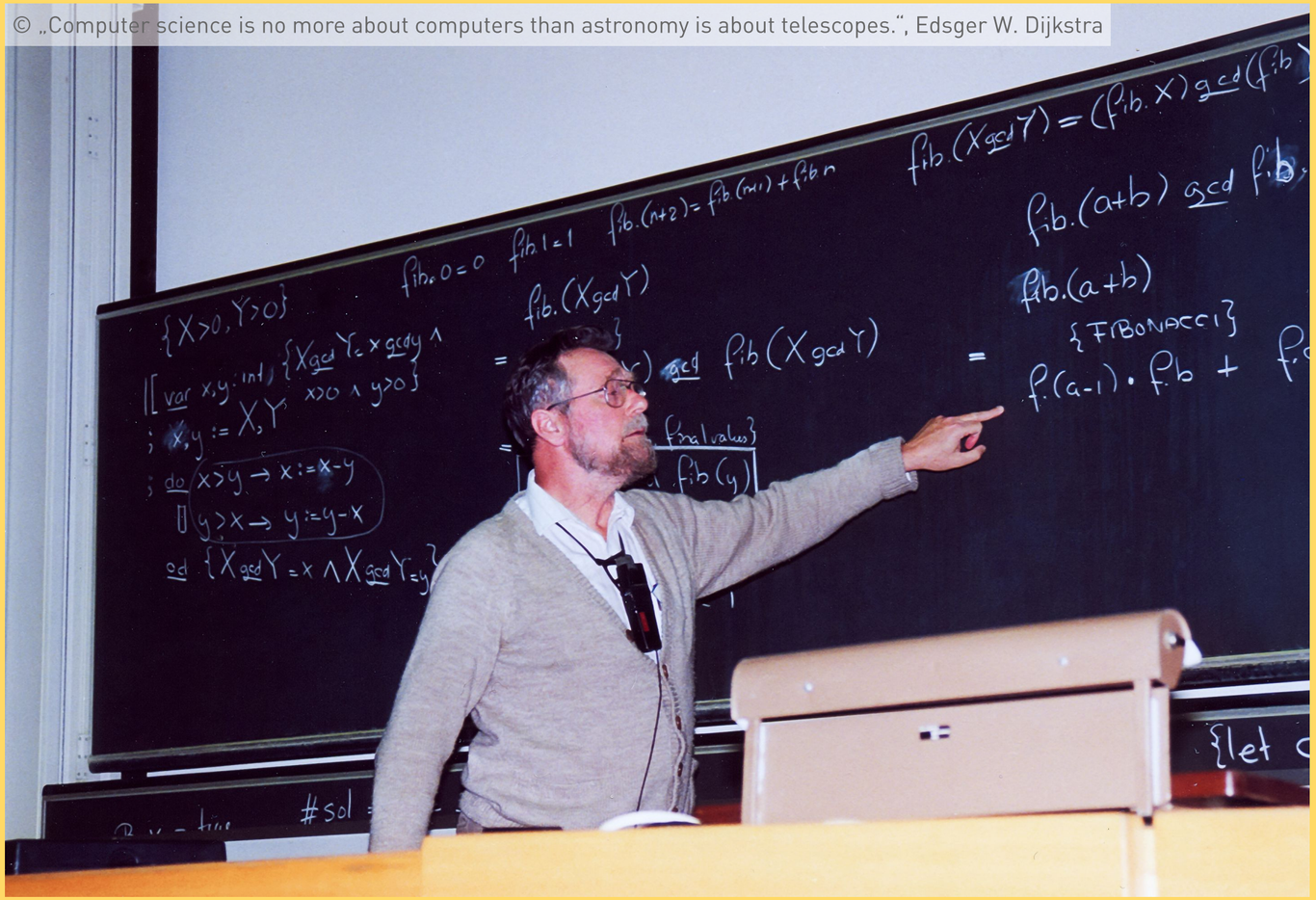
SOFTWAREARCHITEKTUR

Joern Ploennigs · AI4SC

Midjourney: Soft Architecture

Hörsaalfrage

Was ist „Informatik“?



© „Computer science is no more about computers than astronomy is about telescopes.“, Edsger W. Dijkstra

Wiederholung: Definition - Informatik

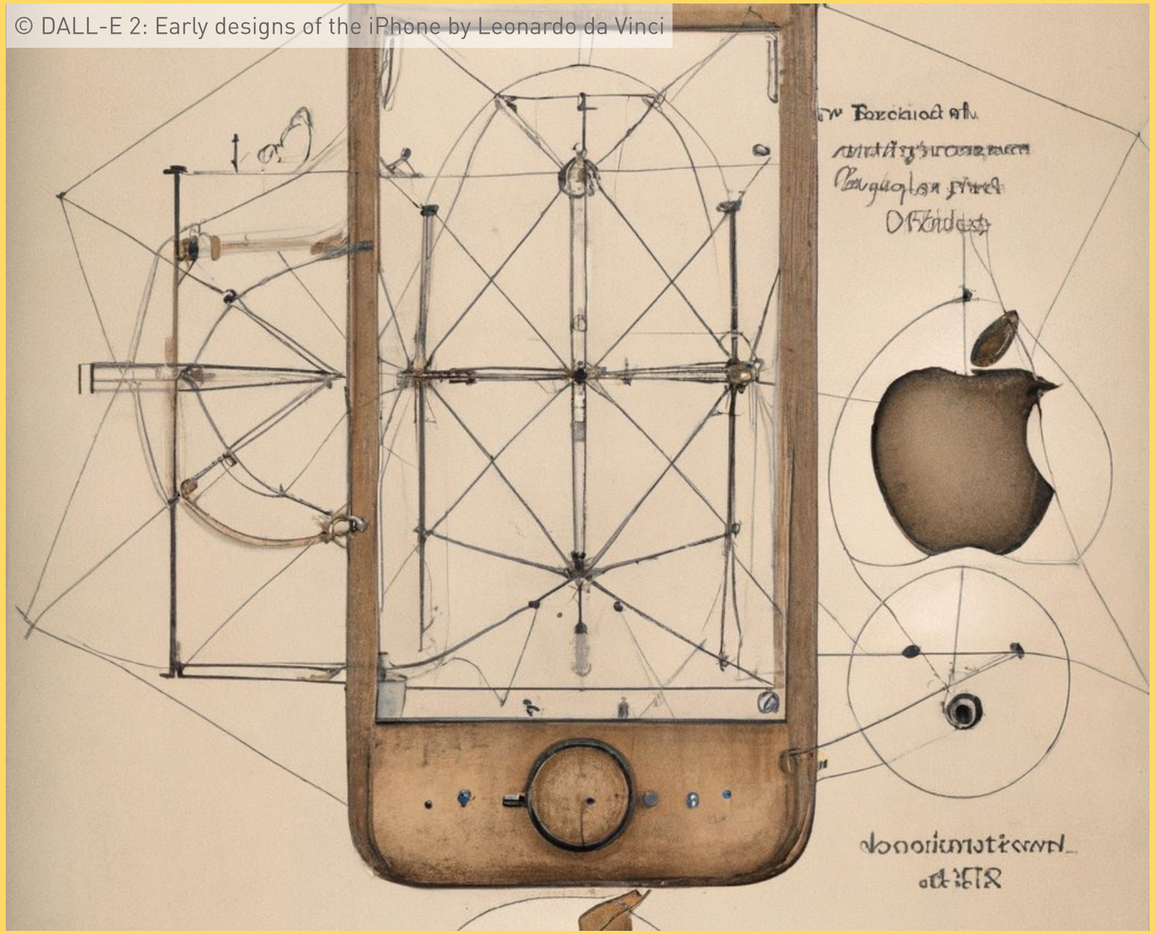
INFORMATIK

Informatik ist die Wissenschaft von der systematischen Verarbeitung von Informationen.

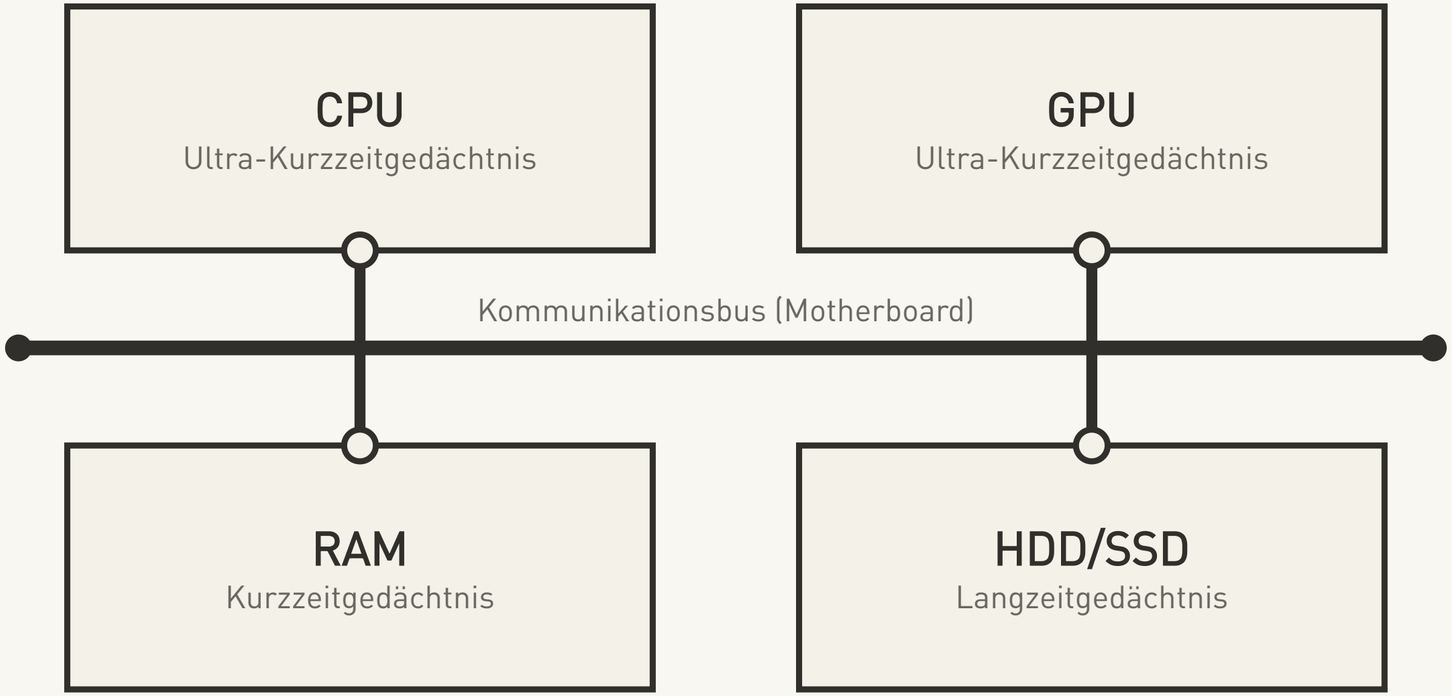
Ursprung: Mathematik (Logik, Algorithmen) + Elektrotechnik (Hardware)

Hörsaalfrage

Aus welcher Hardware besteht ein Computer?


© DALL-E 2: Early designs of the iPhone by Leonardo da Vinci

Wiederholung: Computer Aufbau



Hörsaalfrage

Was sind binäre Zahlen?



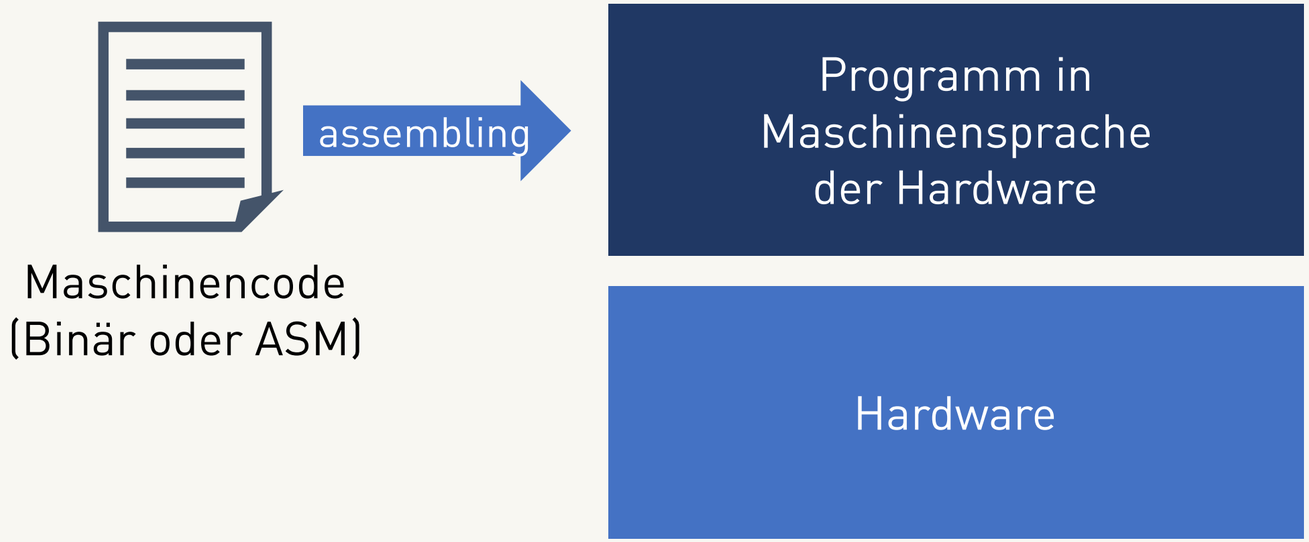
Binärzahlen

- Computer speichern, verarbeiten und kommunizieren Daten als Binärzahlen, weil Transistoren in einem Gatter nur logische Operationen ausführen können
- binarius – Zweifach, doppelt
- Das kleinste mögliche (nützliche) System von Zeichen
- Mögliche Repräsentationen von Binärcode:
  - 0 / 1
  - Falsch / wahr
  - Aus / An

Architektur entscheidet, was Code kann

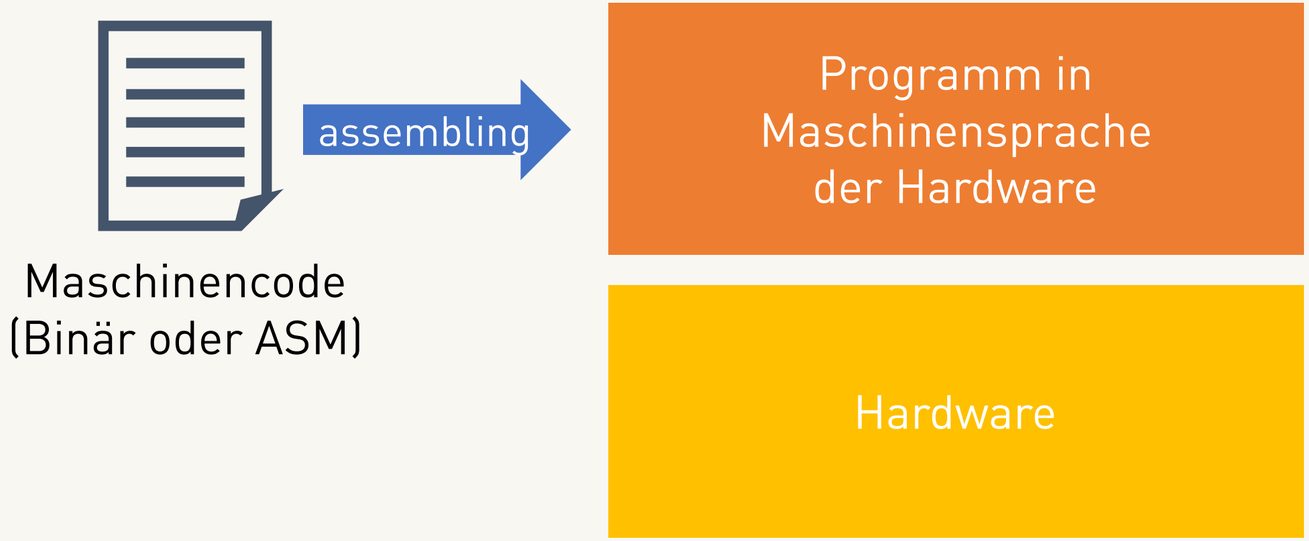
- Jede Ära zeigt dasselbe Muster: Anforderungen überwachsen die bestehende Architektur.
- Die Antwort war eine neue Architektursicht mit neuen Komponentenbeziehungen ⇆ , nicht besserer Code im alten System.
- Die Informationsschicht der Wissenspyramide (→ `02a_Wissenspyramide`) ist die Architektur.

Monolithische, maschinennahe Software vor 1950



- Die ersten Programme wurden direkt im Maschinencode geschrieben (binär oder Assembler).
- Die Software war monolithisch: keine Trennung von Funktionalitäten.

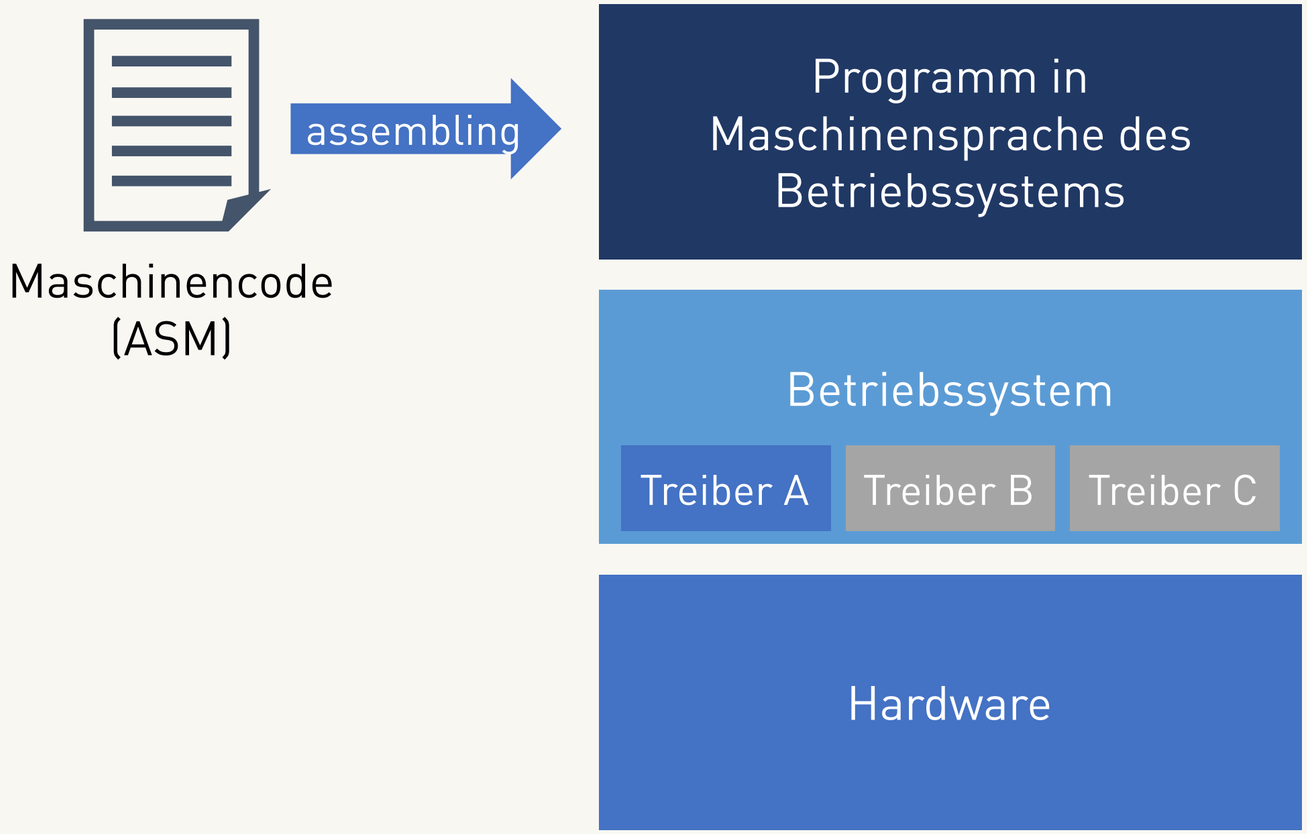
Probleme monolithischer, maschinennaher Software vor 1950



Maschinencode
(Binär oder ASM)

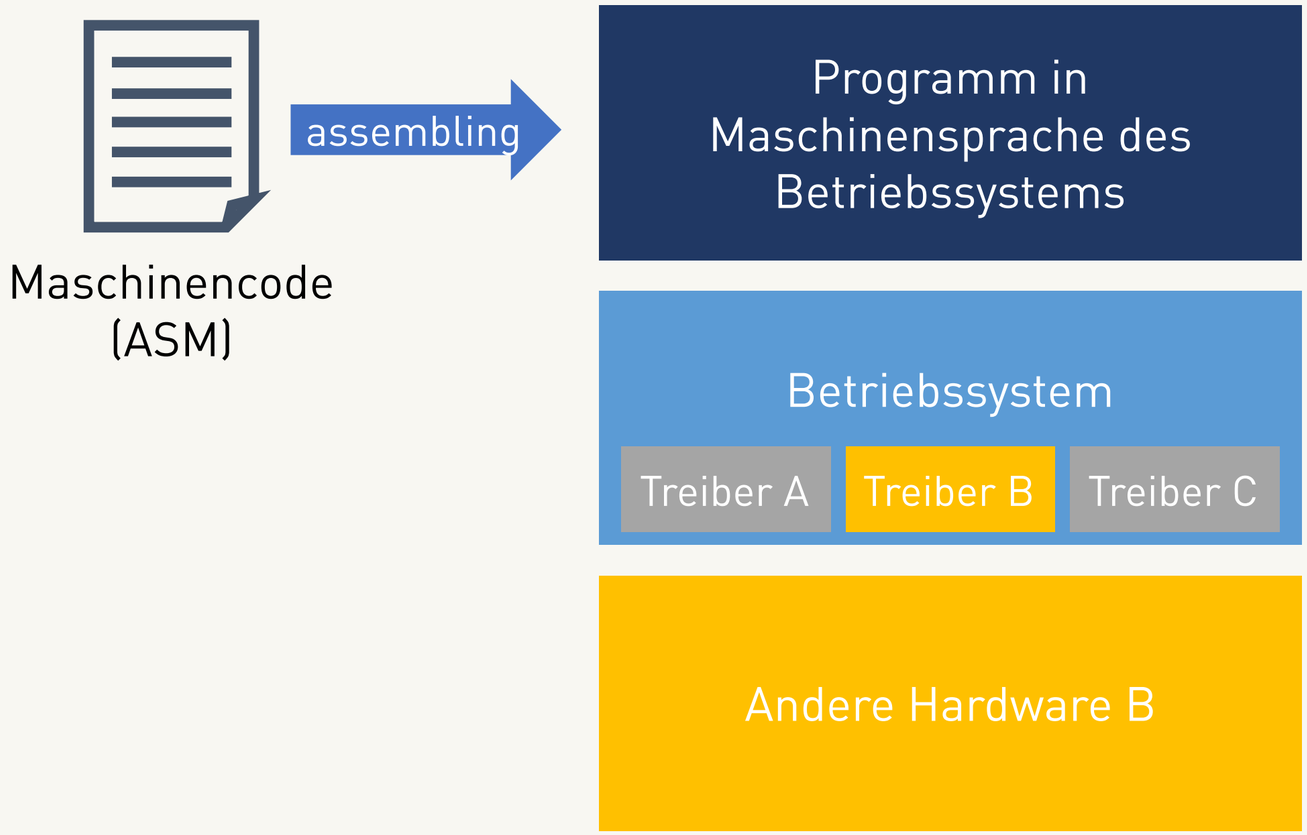
Wenn sich die Hardware änderte und eine neue Maschinensprache gesprochen wurde, mussten Programme angepasst oder neu geschrieben werden.

Aufteilung in Betriebssysteme und monolithische Software ab ca. 1960



- Betriebssysteme übernehmen Standardaufgaben (z. B. Dateiverwaltung) und abstrahieren die Hardware.
- Die Abstraktion erfolgt über Treiber.

Probleme der Aufteilung in Betriebssysteme und Software ab ca. 1960



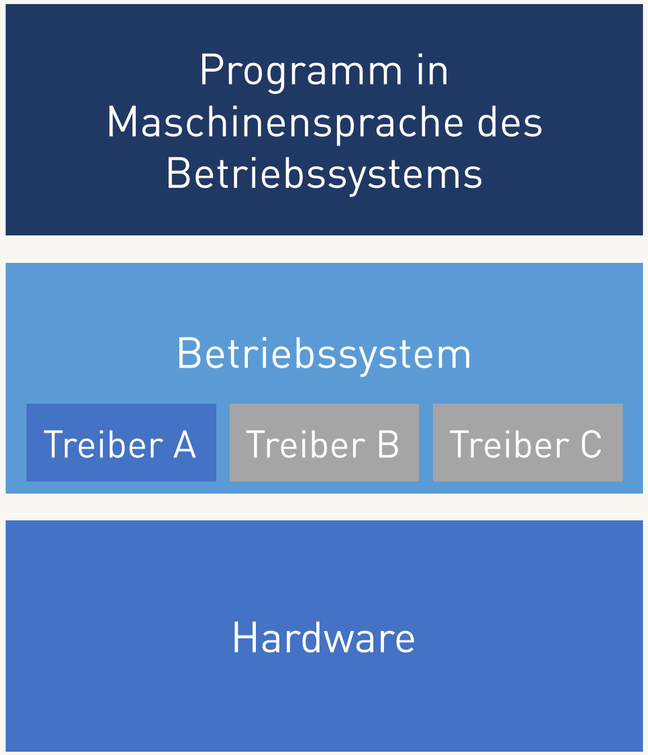
Neue Hardware wird über einen neuen Treiber unterstützt, das Programm muss nicht angepasst werden. Die Programme bleiben aber schwer zu schreiben, weil sie ein gutes Verständnis der Hardware erfordern.

Einführung höherer Programmiersprachen ab ca. 1970



Höhere Programmiersprache (Fortran, C, Basic)

compiler



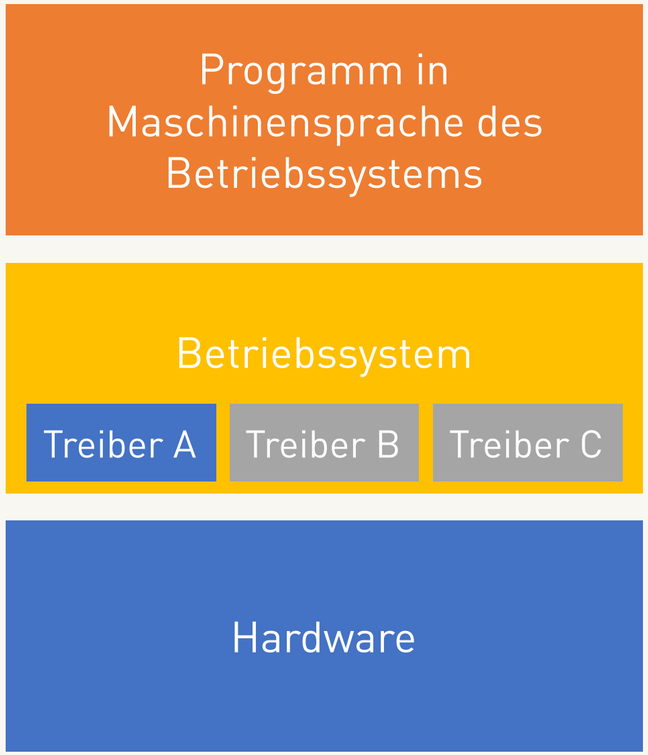
- Höhere Programmiersprachen (Fortran, C, Basic) vereinfachen die Programmierung.
- Ein Compiler übersetzt den Quellcode in Maschinensprache des Betriebssystems.

Probleme mit höheren Programmiersprachen



Höhere Programmiersprache (Fortran, C, Basic)

compiler



Wenn sich das Betriebssystem ändert, funktionieren Programme nicht mehr und müssen neu kompiliert oder umgeschrieben werden.

Virtualisierung monolithischer Software ab ca. 1995



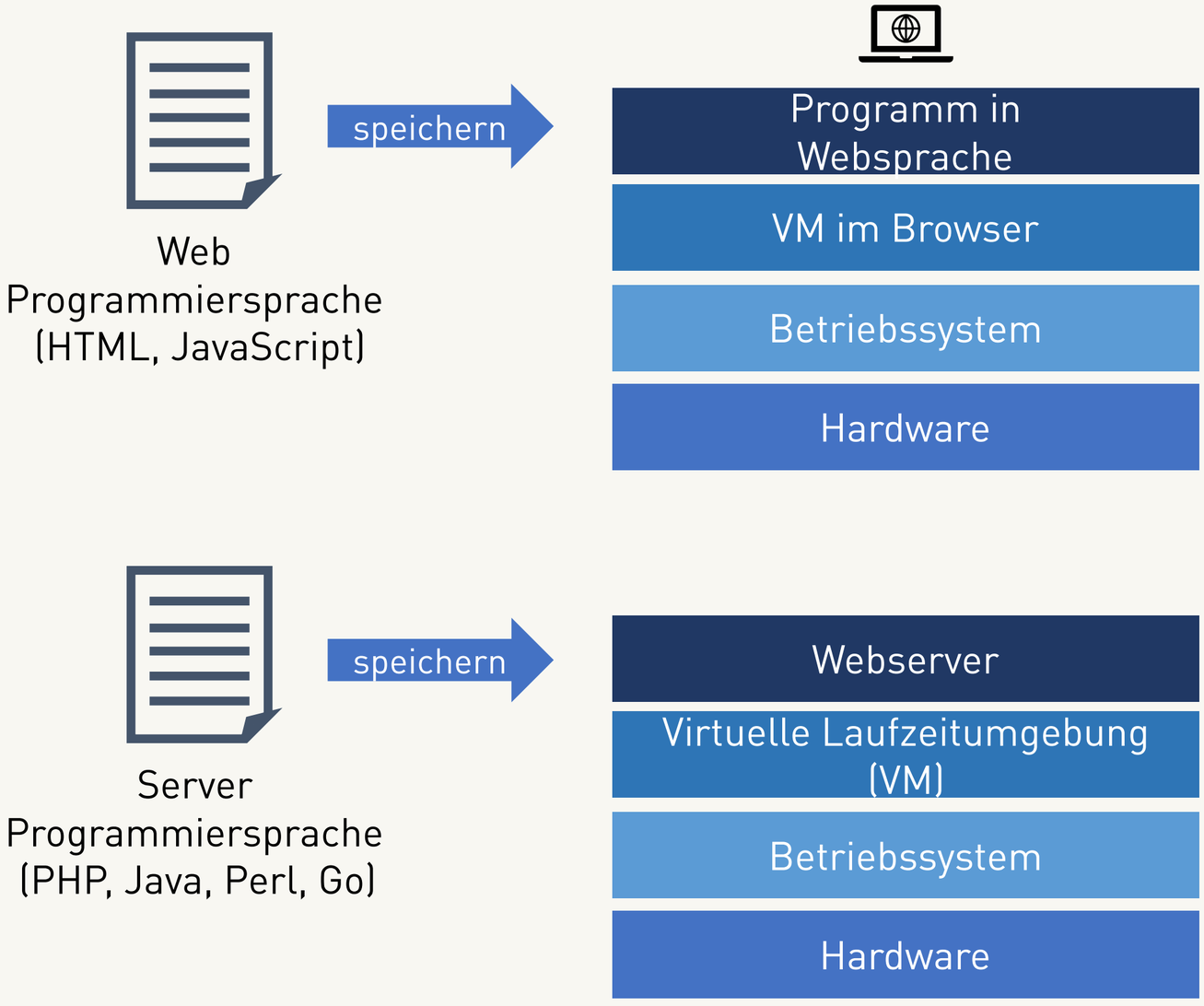
Virtualisierte Programmiersprache (Java, Python)

speichern



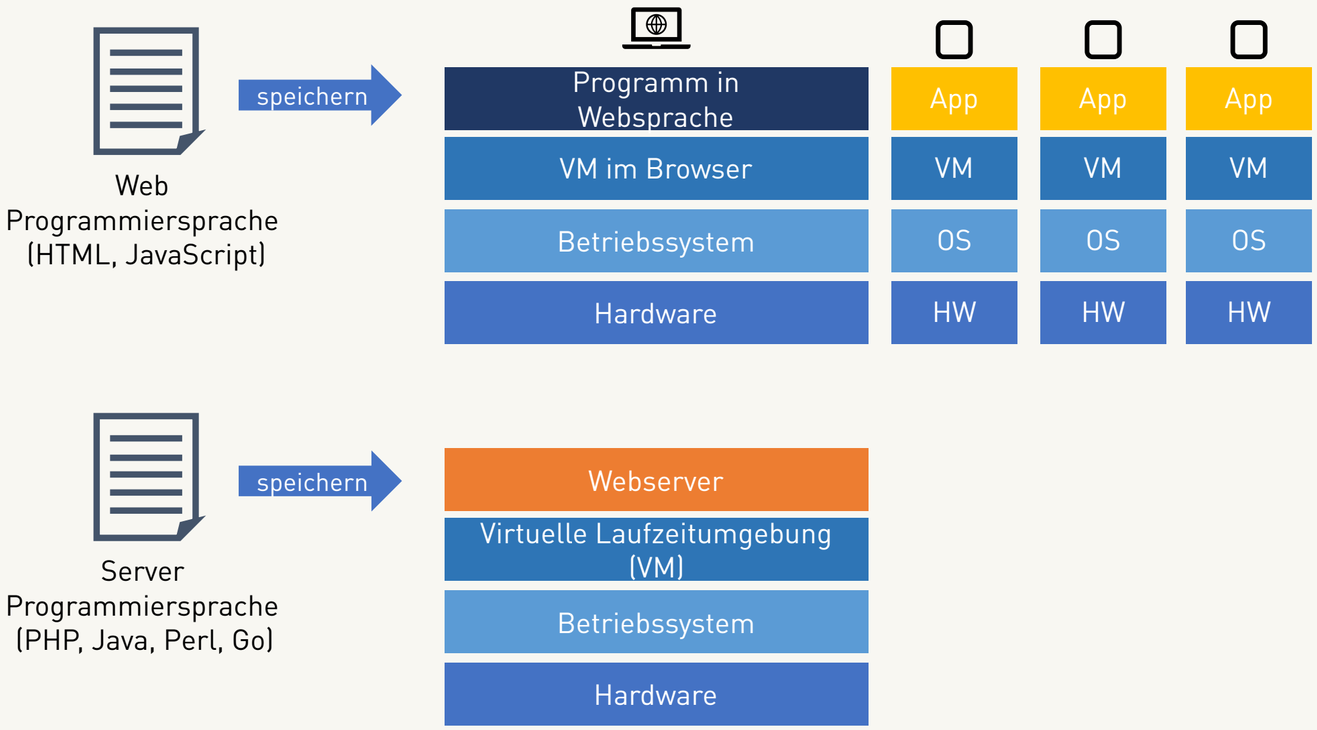
- Virtuelle Laufzeitumgebungen abstrahieren die Betriebssysteme.
- Sie kompilieren den Code in Echtzeit (Java) oder interpretieren ihn (Python).
- Mit dem Internet entstand der Bedarf, auf Inhalte mobil zuzugreifen.

Verteilte, virtualisierte Software im Internet ab ca. 2000



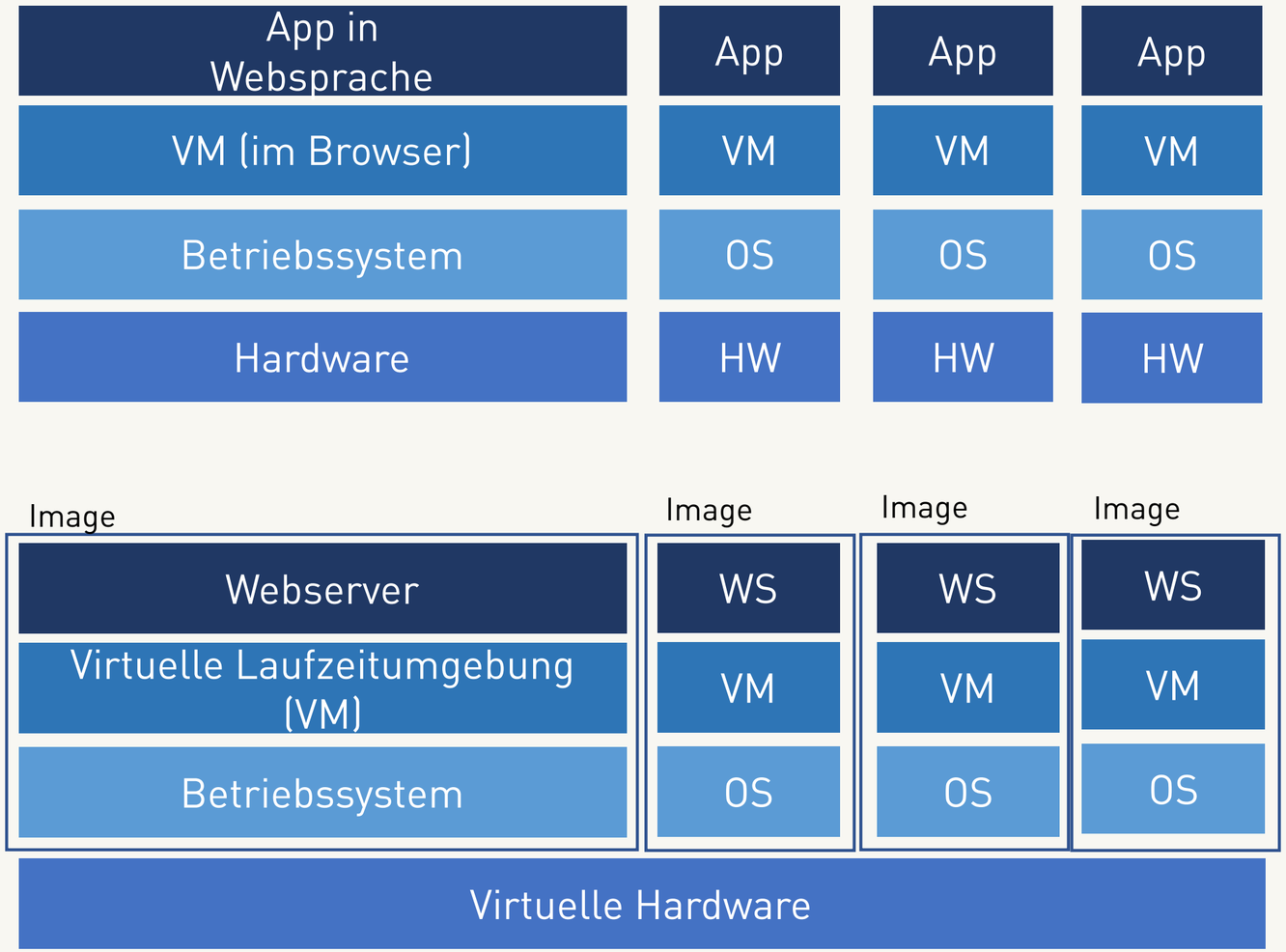
- Frontend-Webseiten visualisieren die Inhalte und interagieren mit den Nutzern.
- Backend-Webserver stellen die Webseiten bereit und verwalten die Langzeitdaten in Datenbanken.

Probleme mit verteilter, virtualisierter Software



Im Internet greifen sehr viele Nutzer gleichzeitig auf dieselben Server zu. Um diese nicht zu überlasten, braucht man Wege, sie bei Bedarf schnell zu duplizieren.

Verteilte, virtualisierte Apps und Cloud-Software ab ca. 2010



- Programme werden mit dem Betriebssystem in Images gebündelt.
- Images lassen sich auf virtueller Hardware schnell auf viele Rechner verteilen.
- Die resultierenden modernen verteilten Anwendungen weisen eine sehr hohe Komplexität auf.

ARCHITEKTUR IM AGENTISCHEN ZEITALTER

KI generiert Komponenten — Ingenieure entscheiden Architektur

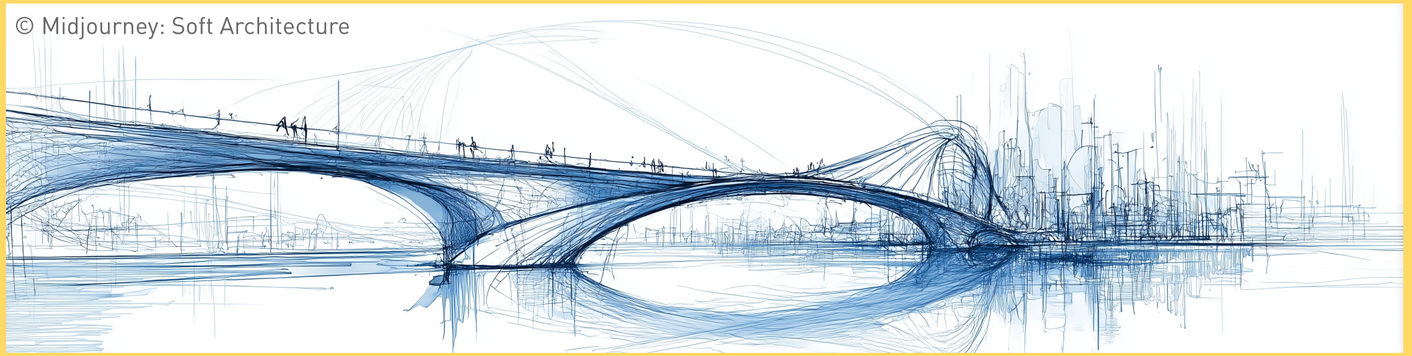
- KI generiert: Funktionen · Klassen · Module · Algorithmen
- KI entscheidet **nicht**: Beziehungen zwischen Komponenten ⇆
- Architekturentscheidungen überdauern den individuellen Code
- *Die Modulgrenze im Prompt ist eine Architekturentscheidung — sie wird von Ingenieuren festgelegt.*

KI generiert: Funktion, Klasse, Modul.

Ingenieur entscheidet: Grenzen, Beziehungen, Verantwortung.

Hörsaalfrage

Warum kann KI keine kompletten Systeme entwerfen?



Prompt-Grenzen sind Architekturgrenzen

- Jede Komponentenbeschreibung im Prompt definiert eine Architekturgrenze
- Falscher Prompt → falsche Architektur → Code schwer zu ändern
- Architektur entsteht nicht nach dem Programmieren — sie entsteht vor dem Prompt
- → `03c` : UML und Prozessmodelle — Architektur vor dem Prompt

Agentic Workflow

Vor dem Prompt:

01 Welche Komponenten?

02 Welche Grenzen?

03 Wie kommunizieren sie miteinander?

04 Dann getrennt generieren.

QUESTIONS